

Modelling Distributed Systems

Process Algebra

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



- 1 Prologue
- 2 Basics
- 3 Operators, Laws, Transitions
- 4 Conclusion



Next in Line...

- 1 Prologue
- 2 Basics
- 3 Operators, Laws, Transitions
- 4 Conclusion



Proving Features of Distributed Systems

Are software/system architectures enough?

- properties are not really proven without some well-defined *formal representation*
- also, software/system architectures model *structure & organisation*, not really *behaviour*
- behavioural properties of distributed systems can be understood qualitatively—*not demonstrated*, really



Focus on Concurrency

Concurrency

- *concurrency* is the first source of complexity in distributed systems
- a possible path is using *formal models for concurrent systems*
- abstracting away from physical location / distribution—in the same way as with software architecture, yet in a more formal way



Harnessing Complexity of Concurrent Systems

Formal models for concurrent systems

- are aimed at capturing the *essence* of concurrent & distributed systems
- *abstracting* away from unnecessary details
- providing *non-ambiguous representation* of computational entities and their behaviour
- as well as the chance to **compute** *properties*
- several approaches exist: in the following, we focus on **process algebra**

Next in Line...

- 1 Prologue
- 2 Basics**
- 3 Operators, Laws, Transitions
- 4 Conclusion



Basics^[Bergstra et al., 2001]

Basic concepts

process — a series of *actions* or *events*

algebra — a *calculus* of *symbols* that combine according to certain defined laws

calculus — a system or method of *calculation*

Process algebra

A **process algebra** is a formal description technique for complex computer systems, especially those with communicating concurrently-executing components

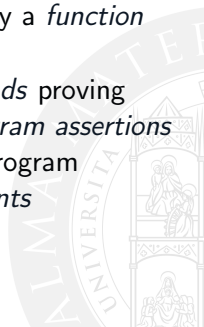
Before Process Algebra^[Baeten, 2005] |

- before process algebra: basically, only Petri Nets existed to reason about concurrent systems^[Peterson, 1977]
- formal reasoning was mostly focussed on giving **semantics** (meaning) to programming languages



Before Process Algebra^[Baeten, 2005] II

- operational semantics** — a computer program is modelled as an *execution* of an *abstract machine*; a *state* of such a machine is a valuation of variables, a *transition* between states is an elementary program instruction
- denotational semantics** — more abstract than operational semantics, computer programs are usually modelled by a *function* transforming *input into output*
- axiomatic semantics** — emphasis is put on *proof methods* proving programs correct: central notions are *program assertions* (proof triples consisting of precondition, program statement, and postcondition) and *invariants*



Process Algebras^[Bergstra et al., 2001] |

- a number of process algebras have been developed
- best known examples being

ACP — Algebra of Communicating Processes^[Bergstra and Klop, 1984]

CCS — Calculus of Communicating Systems^[Milner, 1980]

CSP — Communicating Sequential Processes^[Brookes et al., 1984]



Process Algebras^[Bergstra et al., 2001] II

Basic ingredients

All process algebras share the same key ingredients

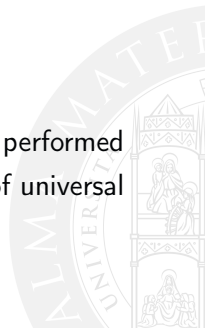
compositional modelling — process algebras provide a small number of constructs for building larger systems up from smaller ones

operational semantics — process algebras are typically equipped with *structural operational semantics* (SOS)^[Plotkin, 1981] describing the single-step execution capabilities of systems, and allowing systems represented as terms in the algebra to be translated into *labelled transition systems* (LTS)

behavioural reasoning — process algebras also feature the use of behavioural relations as a means for relating different systems given in the algebra^[Tribastone, 2014]

Axioms^[Baeten, 2005]

- the word *algebra* denotes an algebraic/axiomatic approach about system behaviour
- that is, process algebras use the methods and techniques of *universal algebra*^[MacLane and Birkhoff, 1967]
- a process algebra is any *mathematical structure* satisfying the **axioms** given for the basic operators
- a process is an *element* of a process algebra
 - in Italian maybe clearer as *algebra dei processi*
- by using the axioms, *calculations with processes* can be performed
- ! often, however, process algebra crosses the boundaries of universal algebra



History^[Baeten, 2005]

- early models focussed on behaviour as *input/output function*
- a process is modelled as an *automaton* in automata theory—an automaton has a number of states and a number of transitions, going from one state to another state
- ? missing what?
- ! *interaction, systems*
 - so process algebra deals with *interacting systems*
- dealing with *concurrent processes*

Process algebra

Process algebra can be seen the *study* of the behaviour of concurrent/distributed systems *by algebraic means*

Next in Line...

- 1 Prologue
- 2 Basics
- 3 Operators, Laws, Transitions**
- 4 Conclusion



Basic Operators^[Baeten, 2005]

A simple set of operators

Operators for combining processes into *systems*

- + *alternative composition*: $x + y$ represents the process executing either x or y
- ; *sequential composition*: $x ; y$ represents the process executing first x then y
 - sequential composition is often represented as $\cdot$ instead: $x \cdot y$
- || *parallel composition*: $x || y$ represents the process executing x and y concurrently / in parallel

Basic Laws of Process Algebra [Baeten, 2005] |

- given a set of **atomic actions** – representing *indivisible behaviour* –, basic operators are used to build more articulated processes
- operators may have *neutral elements*—yet we ignore them for now
- *precedence* typically goes from $+$ (weakest) to $;$

Structural laws

$x + y = y + x$ *commutativity* of alternative composition

$x + (y + z) = (x + y) + z$ *associativity* of alternative composition

$x + x = x$ *idempotency* of alternative composition

$(x + y); z = x; z + y; z$ *right distributivity* of $+$ over $;$

$(x; y); z = x; (y; z)$ *associativity* of sequential composition

$x \parallel y = y \parallel x$ *commutativity* of parallel composition

$(x \parallel y) \parallel z = x \parallel (y \parallel z)$ *associativity* of parallel composition

Basic Laws of Process Algebra^[Baeten, 2005] II

Laws as axioms

Any mathematical structure

- with three binary operations
- satisfying the above laws

is a process algebra

Transition systems

Most typically, such structures are expressed in terms of automata, mostly called *transition systems*

Uses I

↓ [Baeten, 2005]

- process algebra offers a way to *represent* concurrent systems – their structure and behaviour – so they can be used for system **specification**
- process algebra offers a way to *calculate* systems' behaviour – so they can be used for system **verification**

Specification

How do we specify a model for a concurrent / distributed system S with process algebra?

- we (possibly) select the most suitable process algebraic framework, and the specific structural laws
- we define the *atomic actions* of S
- we define the behaviour of the system in terms of *transitions* that comply with the structural laws of algebra

Uses II

Verification

How do we exploit a process algebra to *verify* some features of a model for a concurrent / distributed system S ?

- we exploit axioms and transactions to derive properties for S



Choosing the Laws I

- the seven laws presented before could be extended with *left distributivity* of $+$ over $;$;
- what is the effect of adding / removing that law?

Process algebra with left distributivity

$x + y = y + x$ *commutativity* of alternative composition

$x + (y + z) = (x + y) + z$ *associativity* of alternative composition

$x + x = x$ *idempotency* of alternative composition

$(x + y); z = x; z + y; z$ *right distributivity* of $+$ over $;$;

$x; (y + z) = x; y + x; z$ *left distributivity* of $+$ over $;$;

$(x; y); z = x; (y; z)$ *associativity* of sequential composition

$x \parallel y = y \parallel x$ *commutativity* of parallel composition

$(x \parallel y) \parallel z = x \parallel (y \parallel z)$ *associativity* of parallel composition

Choosing the Laws II

Branching vs. linear time

- if we adopt *both* right and left associativity of $+$ over $;$ then the moment of choice does not matter: this is called *linear time*
- otherwise, the moment of choice *does* matter: this is called *branching time*



Transitions I

- each atomic action a is a constant that can execute itself, after which it terminates successfully ($\checkmark$)

$$\frac{}{a \xrightarrow{a} \checkmark}$$

- alternative composition can proceed either way

$$\frac{x \xrightarrow{a} \checkmark}{x+y \xrightarrow{a} \checkmark} \quad \frac{x \xrightarrow{a} x'}{x+y \xrightarrow{a} x'} \quad \frac{y \xrightarrow{a} \checkmark}{x+y \xrightarrow{a} \checkmark} \quad \frac{y \xrightarrow{a} y'}{x+y \xrightarrow{a} y'}$$

- sequential composition just proceeds one way

$$\frac{x \xrightarrow{a} \checkmark}{x;y \xrightarrow{a} y} \quad \frac{x \xrightarrow{a} x'}{x;y \xrightarrow{a} x';y}$$



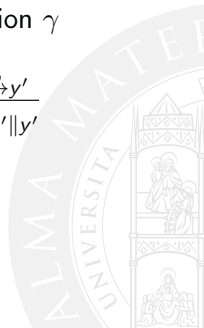
Transitions II

- parallel composition may proceed either simply

$$\frac{x \xrightarrow{a} \checkmark}{x \parallel y \xrightarrow{a} y} \quad \frac{x \xrightarrow{a} x'}{x \parallel y \xrightarrow{a} x' \parallel y} \quad \frac{y \xrightarrow{a} \checkmark}{x \parallel y \xrightarrow{a} x} \quad \frac{y \xrightarrow{a} y'}{x \parallel y \xrightarrow{a} x \parallel y'}$$

or with processes interacting via a communication function γ

$$\frac{x \xrightarrow{a} \checkmark \quad y \xrightarrow{b} \checkmark}{x \parallel y \xrightarrow{\gamma(a,b)} \checkmark} \quad \frac{x \xrightarrow{a} x' \quad y \xrightarrow{b} \checkmark}{x \parallel y \xrightarrow{\gamma(a,b)} x'} \quad \frac{x \xrightarrow{a} \checkmark \quad y \xrightarrow{b} y'}{x \parallel y \xrightarrow{\gamma(a,b)} y'} \quad \frac{x \xrightarrow{a} x' \quad y \xrightarrow{b} y'}{x \parallel y \xrightarrow{\gamma(a,b)} x' \parallel y'}$$



Operational vs. Axiomatic

Overall, roughly speaking

- *structural laws* represent the axioms allowing system properties to be proven
- *transition rules* provide an operational representation suitable as a reference for implementation
- when transitions are defined such as to abide by structural laws, in principle we have *implementation with formal properties*



Next in Line...

- 1 Prologue
- 2 Basics
- 3 Operators, Laws, Transitions
- 4 Conclusion**



Lessons Learnt I

For good

- formal models are essential in distributed systems for proving properties, and for constraining implementation of components and systems
- process algebras are among the most widely used techniques for representing distributed systems and reasoning about their properties
- it is not so difficult to handle them to represent some basic properties
- proof can be built over systems

Lessons Learnt II

For not-so-good

- proofs are not so intuitive—at least not enough to be an everyday tool for software engineers
- we just dealt with concurrent computation
 - not a mention to *physical distribution*
- so, we now know how to deal with concurrent systems – at least in principle –, not with distributed systems
 - there are formal techniques dealing with that, of course, but we call ourselves content with process algebra, as far as this course is concerned

Modelling Distributed Systems

Process Algebra

Distributed Systems

Andrea Omicini
andrea.omicini@unibo.it

Dipartimento di Informatica – Scienza e Ingegneria (DISI)
ALMA MATER STUDIORUM – Università di Bologna

Academic Year 2025/2026



References I

- [Baeten, 2005] Baeten, J. C. (2005).
A brief history of process algebra.
Theoretical Computer Science, 335(2-3):131–146
 DOI:10.1016/j.tcs.2004.07.036
- [Bergstra and Klop, 1984] Bergstra, J. A. and Klop, J. W. (1984).
Process algebra for synchronous communication.
Information and Control, 60(1-3):109–137
 DOI:10.1016/S0019-9958(84)80025-X
- [Bergstra et al., 2001] Bergstra, J. A., Ponse, A., and Smolka, S. A., editors (2001).
Handbook of Process Algebra.
 Elsevier Science B.V., 1st edition
<https://www.elsevier.com/books/handbook-of-process-algebra/bergstra/978-0-444-82830-9>
- [Brookes et al., 1984] Brookes, S. D., Hoare, C. A. R., and Roscoe, A. W. (1984).
A theory of communicating sequential processes.
Journal of the ACM, 31(3):560–599
 DOI:10.1145/828.833
- [MacLane and Birkhoff, 1967] MacLane, S. and Birkhoff, G. (1967).
Algebra.
 MacMillan, New York, NY, USA
- [Milner, 1980] Milner, R., editor (1980).
A Calculus of Communicating Systems, volume 92 of *Lecture Notes in Computer Science*.
 Springer Berlin Heidelberg, Berlin, Heidelberg
 DOI:10.1007/3-540-10235-3



References II

[Peterson, 1977] Peterson, J. L. (1977).

Petri Nets.

ACM Computing Surveys, 9(3):223–252

(APICe) DOI:10.1145/356698.356702

[Plotkin, 1981] Plotkin, G. D. (1981).

A structural approach to operational semantics.

Technical report, Computer Science Department, Aarhus University

[Tribastone, 2014] Tribastone, M. (2014).

Behavioral relations in a process algebra for variants.

In *18th International Software Product Line Conference (SPLC '14)*, volume 1, pages 82—91, New York, NY, USA. ACM

DOI:10.1145/2648511.2648520

